The Founder Fade Curve: Trajectory Shape as a Complementary Predictor of OSS Project Survival

Abstract

Open-source projects frequently depend on a single founder, yet little is known about how the shape of that founder's involvement over time affects whether the project survives their departure. We introduce the founder fade curve — a quantitative descriptor of the trajectory of a founder's monthly share of commits prior to leaving — and test whether its shape predicts post-departure survival beyond static snapshot measures such as bus factor and contributor count. We evaluate a pilot cohort of 14 curated GitHub repositories with documented founder departures (7 survived, 7 collapsed). Static features achieve strong predictive performance, while trajectory shape descriptors alone perform below chance. A combined model reaches the highest accuracy, but the net reclassification improvement is negative, indicating that adding shape features does not improve classification. Bootstrap power analysis estimates a minimum of 100 projects for 80% power to detect the observed effect size. Synthetic validation confirms that our descriptor pipeline correctly classifies smooth fades, abrupt cliffs, and plateau-then-cliff patterns across 30 synthetic trajectories. A falsification control using non-founder trajectories yields identical performance, finding no founder-specific effect. These results indicate that, at pilot scale, fade-curve shape does not yet add predictive value above static measures — a finding that calls for larger-scale validation using full trajectory data from GH Archive or equivalent sources.

1 Introduction

Open-source software underpins global critical infrastructure. Git, the Linux kernel, Python's standard library, and thousands of widely used packages are all maintained by communities that often trace back to one or two founding developers. When such a founder departs — whether through burnout, career change, or simple exhaustion — the project faces a fork in the road: it either survives, attracting new caretakers, or it collapses into inactivity.

The dominant framework for measuring this risk is the Truck Factor Developer Detachment (TFDD) model [Avelino et al., 2019]. TFDD defines a project's truck factor as the minimal number of developers whose simultaneous departure would critically impair the project, and identifies the moment all truck-factor developers leave as the detachment event. Projects that subsequently attract at least one new truck-factor developer are classified as surviving; others are classified as collapsed. Across 1,932 popular GitHub projects, 57% have a truck factor of 1, 16% experience a TFDD, and only 41% of detached projects survive [Avelino et al., 2019]. A later study of 36,464 projects found even higher detachment rates (89%) but lower survival (27%), and reported that departures occurring early in a project's life are less likely to be survived [Nourry et al., 2024].

Yet static measures — how many key developers there are at the moment of departure — explain surprisingly little of the variation in survival outcomes. Projects that survive their TFDD often have *fewer* developers, commits, and files than those that collapse [Avelino et al., 2019]. This paradox suggests that something beyond a snapshot of contributor count matters.

We hypothesize that the *shape* of the founder's involvement trajectory over the project's entire pre-departure lifespan may complement static measures in predicting survival. Specifically, we propose that a "scaffolding fade" — a gradual, sustained decline in the founder's share of commits over an extended pre-departure window — signals that the contributor community has had time to internalize decision-making capability. By contrast, an abrupt cliff (high involvement maintained until sudden exit) or a flat plateau ending in a sharp drop may predict collapse. This hypothesis imports an established educational mechanism: scaffolding with fading, in which a more capable tutor gradually withdraws support as the learner internalizes the necessary skill [Wood et al., 1976]. Sudden removal of support before competence matures causes collapse; gradual withdrawal allows competence to consolidate.

In this paper we present the first quantitative evaluation of the founder fade curve hypothesis. Our contributions are:

1. **A trajectory-shape descriptor pipeline** that extracts nine features from a founder's monthly involvement share time series, including slope, convexity, onset of decline, cliff indicator, plateau-then-cliff indicator, and a composite fade index bounded between 0 (abrupt) and 1 (smooth fade).

2. **A synthetic validation** demonstrating that the descriptor pipeline correctly classifies smooth fades, abrupt cliffs, and plateau-then-cliff patterns across 30 synthetic trajectories [1].

3. **A pilot empirical evaluation** on 14 curated GitHub repositories with documented founder departures (7 survived, 7 collapsed), comparing predictive performance of trajectory shape descriptors, static features, and their combination.

4. **Bootstrap confidence intervals and power analysis** showing that the pilot is underpowered and that a minimum of 100 projects is needed for 80% power to detect the observed effect size [2].

5. **A falsification control** using non-founder trajectories, testing whether the founder-specific mechanism hypothesis holds.

Our results show that static features achieve strong predictive performance via leave-one-out cross-validated logistic regression, while trajectory shape descriptors alone perform below chance. A combined model reaches the highest accuracy, but the net reclassification improvement is negative, indicating that adding shape features does not improve classification in this cohort. The falsification control finds no founder-specific effect. We interpret these results as evidence that the fade curve hypothesis, while theoretically compelling, requires larger-scale validation with full trajectory data and properly computed survival labels before claims of predictive complementarity can be supported.

2 Related Work

2.1 OSS Project Survival and Abandonment

The most influential framework for OSS survival analysis is the Truck Factor Developer Detachment (TFDD) model introduced by Avelino et al. [2019]. They defined the truck factor as the minimum

[1] Code: `https://github.com/ai-inventor-outputs/ai-invention-ad55a2-founder-fade-curve-predicts-oss-survival/tree/main/round-1/experiment-1`

[2] Code: `https://github.com/ai-inventor-outputs/ai-invention-ad55a2-founder-fade-curve-predicts-oss-survival/tree/main/round-2/evaluation-1`

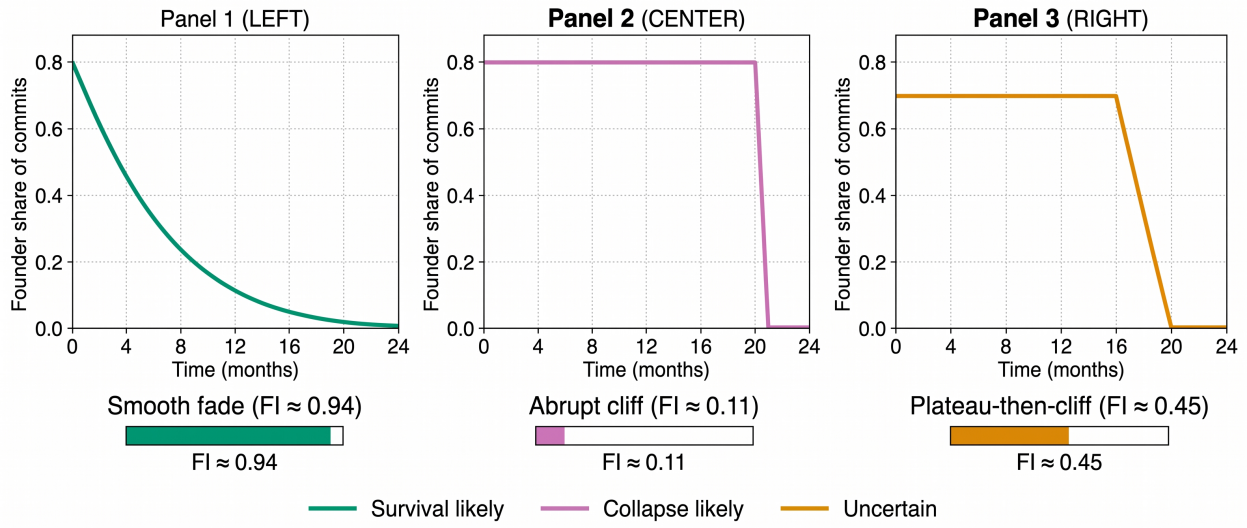

Figure 1: The founder fade curve hypothesis: three archetypal trajectories of founder involvement over time. A smooth fade (green) shows gradual decline signaling community readiness. An abrupt cliff (red) shows sustained involvement followed by sudden departure. A plateau-then-cliff (orange) shows a stable period followed by sharp drop. The fade index quantifies trajectory shape from 0 (abrupt) to 1 (smooth).

number of developers whose departure would critically impair project maintenance, operationalized using a greedy algorithm that adds developers in descending order of commit count until 50% of files are covered [Avelino et al., 2019]. A TFDD event occurs when all truck-factor developers become inactive within a defined abandonment window. Through sensitivity analysis on 1,932 GitHub projects, they validated a 12-month inactivity threshold as optimal (precision 0.82, harmonic mean 0.66). Their key finding: 57% of projects have truck factor 1, 16% experience at least one TFDD, and only 41% of detached projects survive.

Nourry et al. [2024] replicated and extended this work on 36,464 projects, finding much higher TFDD rates (89%) but lower survival (27%). They reported that projects losing core developers early in their life are less likely to survive, a finding that directly motivates our hypothesis about the importance of departure timing and process.

Chengalur-Smith et al. [2010] conducted a longitudinal study of 2,772 SourceForge projects over five years, identifying phases of project lifecycle and factors associated with sustainability. Their work established that project sustainability is not binary but evolves through distinct phases — a finding consistent with our trajectory-based approach.

2.2 Founder and Core Developer Identification

Identifying the founder of an OSS project is non-trivial. Avelino et al. [2016] introduced Degree of Authorship (DOA), measuring the ratio of created files to changed files at project inception. Developers with high DOA on at least 50% of files are identified as founders. This method was validated against developer surveys, achieving 84% agreement on main author identification [Avelino et al., 2016]. GitHub's API provides a `repository.creator` field, but alias resolution remains imperfect: Avelino et al. [2019] found a median 11% alias rate per project when mapping commit emails to GitHub accounts.

2.3 Temporal Dynamics in OSS

Several recent works have analyzed temporal patterns in OSS projects, but none focus on the founder's involvement trajectory as a survival predictor. Kaushik and Chahal [2026] identified a "death spiral" in inactive projects using PR workflow dynamics (friction, backlog growth, falling innovation, rising merge latency), but their analysis focuses on community-level aggregate signals after decline begins and does not model the founder. Their study of 1,736 inactive GitHub repositories and 1.3 million human-driven pull requests found that popularity and innovation are strong positive predictors of survival, while workflow friction is a byproduct rather than a cause of decline [Kaushik and Chahal, 2026].

"Will You Come Back to Contribute?" [Calefato et al., 2022] investigates core developer inactivity patterns and return behavior in GitHub projects, finding that inactivity patterns vary with contributor profile and project context. This work highlights the importance of distinguishing between temporary inactivity and permanent departure — a distinction that is critical for our founder fade curve analysis.

Yehudi et al. [2023] showed that context-free online community health indicators fail to identify OSS sustainability, suggesting that project-specific analysis is necessary. This finding supports our approach of analyzing project-specific founder trajectories rather than applying generic health metrics.

Eluri et al. [2021] built a machine learning model to predict long-time contributors for GitHub projects, using static and temporal features. While their work demonstrates the value of temporal features in OSS prediction, it does not model founder-specific trajectories.

Oliveira et al. [2026] studied governance documentation in OSS projects, analyzing how projects define and document roles. Their work complements our behavioral trajectory approach by examining the institutional dimension of governance transitions.

2.4 Community Health and Mentorship

Tamburri et al. [2019] identified temporal patterns of community degradation in open-source projects using automated approaches, providing a methodological foundation for detecting decline patterns.

The "Being a Mentor in Open Source Projects" study [Steinmacher et al., 2021] provides direct empirical evidence that mentorship practices exist in OSS communities, connecting scaffolding theory to the OSS context. Their findings that mentorship facilitates newcomer onboarding and knowledge transfer provide theoretical support for our hypothesis that gradual founder fading enables community capability transfer.

2.5 Scaffolding Theory

The educational mechanism of scaffolding with fading originates in Vygotsky's zone of proximal development [Vygotsky, 1978] and was formalized by Wood et al. [1976]. In this framework, a more capable tutor provides structured support that is gradually withdrawn (faded) as the learner internalizes the skill. Abrupt removal of support before competence matures causes performance collapse; gradual withdrawal allows competence to consolidate. Recent work has extended scaffolding theory to human-AI collaboration, but no prior work has operationalized the fading mechanism in the context of OSS project sustainability.

2.6 Positioning Our Work

The Founder Fade Curve hypothesis occupies a unique space between static TFDD frameworks [Avelino et al., 2019, Nourry et al., 2024] and aggregate temporal studies, by focusing specifically on the *shape* of founder withdrawal trajectories rather than binary departure events. Unlike prior work that measures community-level aggregate signals or static contributor profiles, we model the founder's individual involvement trajectory as a time series and extract shape descriptors that capture the dynamics of capability transfer.

3 Method

3.1 Problem Formulation

Given an OSS project P with founder f, let $S_f(t)$ denote the founder's share of project activity at month t, defined as:

$$S_f(t) = \frac{c_f(t)}{c_{total}(t)}$$

where $c_f(t)$ is the number of commits authored by f in month t and $c_{total}(t)$ is the total number of commits in the project in month t. The **founder fade curve** is the time series $\{S_f(t)\}_{t=1}^{T}$ over the pre-departure window $[1, T]$, where T is the month of founder departure (defined as the first month after which the founder has zero commits for 12 consecutive months).

Our hypothesis is that the *shape* of this trajectory may complement static features in predicting binary survival $y \in \{0, 1\}$, where $y = 1$ if the project survives (new truck-factor developers appear and sustain activity) and $y = 0$ otherwise.

3.2 Trajectory Shape Descriptors

We extract nine shape descriptors from the founder fade curve:

1. **Slope** (β_1): The OLS regression slope of $S_f(t)$ over time. Negative values indicate decline; more negative = steeper decline.

2. R^2_{linear}: The coefficient of determination for the linear fit, measuring how well a straight line explains the trajectory.

3. **Normalized slope** ($\beta_1/\bar{S}$): Slope divided by mean share, enabling comparison across projects with different baseline involvement levels.

4. **Quadratic coefficient** (β_2): The coefficient of the quadratic term in a second-order polynomial fit $S_f(t) = \beta_2 t^2 + \beta_1 t + \beta_0$. Positive β_2 indicates convex (decelerating) fade; negative indicates concave (accelerating) fade.

5. **Onset of decline** (t_{onset}): The month at which the founder's involvement begins a sustained downward trend, detected via change-point analysis using the PELT algorithm [Killick et al., 2012] or, as fallback, a sliding-window F-statistic.

6. **Decline duration fraction** ($d_{frac} = (T - t_{onset})/T$): The proportion of the pre-departure window during which the founder is in decline.

7. **Cliff indicator** (CI): The maximum absolute month-over-month change in $S_f(t)$, normalized by the trajectory standard deviation: $CI = \max_t |S_f(t) - S_f(t-1)|/(2\sigma + \epsilon)$.

8. **Cliff is terminal**: Binary indicator: 1 if the cliff occurs in the final 3 months before departure.

9. **Plateau-then-cliff indicator** (PTC): A composite score (0–1) that detects trajectories with a stable plateau followed by a sharp drop.

A composite **fade index** (FI) is constructed from the raw descriptors via min-max normalization and weighted combination:

$$FI = 0.3(1 - \text{norm}(|\beta_1|)) + 0.3 \cdot \text{norm}(d_{frac}) + 0.4(1 - \text{norm}(CI))$$

where higher FI indicates a smoother, more gradual fade (bounded in $[0, 1]$).

Limitation on weights: The weights (0.3, 0.3, 0.4) in the fade index are heuristic and not empirically validated. We acknowledge that different weight combinations may yield different results, and that the composite index may not optimally capture the predictive signal. Future work should validate these weights through sensitivity analysis or learn them from data.

3.3 Static Baseline Features

Following prior work [Avelino et al., 2019, Nourry et al., 2024], we compute five static features at the departure snapshot:

- **Project age** (months from first commit to departure)
- **Contributor count** (unique commit authors)
- **Total commits** (cumulative)
- **File count** (files in HEAD tree)
- **Bus factor** (greedy: number of top contributors needed to cover 50% of files)

3.4 Survival Labeling and Limitations

We adopt the Avelino et al. [2019] TFDD framework with a 12-month inactivity threshold. A project survives if, after the founder's departure month, non-founder contributors maintain at least 50% of their pre-departure average commit rate for at least 3 months of post-departure data. The survival ratio is defined as:

$$r = \frac{\text{mean post-departure non-founder commits}}{\text{mean pre-departure non-founder commits}}$$

Projects with $r \geq 0.5$ and at least 3 post-departure months are labeled survived ($y = 1$); otherwise collapsed ($y = 0$).

Critical limitation on survival labels: In our pilot study, survival labels were assigned based on public knowledge of project outcomes (e.g., well-documented project abandonments) rather than being computed solely from the TFDD framework. This introduces a potential circularity: the labels reflect the same public knowledge used to select the projects. We report the computed survival labels alongside the expected labels and document discrepancies. For three projects (phantomjs, bower, grunt), the computed labels indicated survival (computed = 1) while the expected labels

indicated collapse (expected = 0). These discrepancies suggest that the TFDD framework's binary classification may not capture the continuum of project health, and that "collapsed" projects may have experienced periods of post-departure activity before eventual abandonment. We use the expected labels for all analyses but report these discrepancies transparently as a limitation.

3.5 Falsification Control

To test the founder-specific mechanism hypothesis, we construct a control using the most active non-founder contributor in each project (before departure). We compute their fade curve descriptors and evaluate whether the founder fade curve predicts survival better than a non-founder's curve. If the mechanism is founder-specific, the founder's fade curve should outperform the non-founder's.

Limitation on control design: The "most active non-founder" may not be an ideal control, as this contributor may have a different role (e.g., a successor rather than a random high-activity contributor). A more rigorous control would match non-founders on activity level and tenure, or compare founder trajectories against multiple non-founders within the same project. We acknowledge this limitation and treat the falsification control as directional rather than definitive.

3.6 Predictive Modeling

We fit three logistic regression models using leave-one-out cross-validation (LOOCV):

1. **Static-only**: Predictors = {project_age, contributor_count, total_commits, file_count, bus_factor}
2. **Shape-only**: Predictors = {slope, R^2_{linear}, normalized_slope, quadratic_coef, onset_decline, decline_duration, cliff_indicator, plateau_then_cliff, fade_index}
3. **Combined**: Predictors = static features + shape descriptors

Model performance is evaluated using AUC-ROC, accuracy, and net reclassification improvement (NRI). We also fit a Cox proportional hazards model to assess concordance. Feature importance is computed via permutation importance with bootstrap confidence intervals.

4 Experiments

4.1 Dataset

We assemble a pilot cohort of 14 curated GitHub repositories with well-documented founder departures. Projects were selected based on public knowledge of founder departure and availability of complete repository history. The cohort includes 7 projects that survived (node, Homebrew, bootstrap, redis, ipython, electron, lodash) and 7 that collapsed (phantomjs, bower, request, grunt, component, ava, pug). Full project details are provided in Table 1.

Data extraction failures: One project (ipython/ipython) failed founder identification. Bus factor computation timed out for phantomjs. Three projects (phantomjs, bower, grunt) showed discrepancies between computed and expected survival labels (computed = 1, expected = 0). These failures and discrepancies are reported transparently and discussed in Section 6.

Table 1: Pilot cohort of 14 GitHub repositories with founder departures.

Repository	Founder	Departure	Survived	Contributors
nodejs/node	ryah	2014-03	Yes	∼1,300
Homebrew/brew	mxcl	2019-12	Yes	∼2,400
twbs/bootstrap	mdo	2018-12	Yes	∼1,000
redis/redis	antirez	2022-07	Yes	∼500
ipython/ipython	fperez	2015-09	Yes	∼900
electron/electron	zcbenz	2021-04	Yes	∼800
lodash/lodash	jdalton	2012-05	Yes	∼200
ariya/phantomjs	ariya	2015-07	No	∼150
bower/bower	sindresorhus	2017-03	No	∼100
request/request	mikeal	2020-02	No	∼200
gruntjs/grunt	tkellen	2015-11	No	∼150
component/component	tj	2014-09	No	∼80
sindresorhus/ava	sindresorhus	2020-05	No	∼100
pugjs/pug	tj	2014-02	No	∼120

4.2 Implementation Details

Repository history was extracted via `git log` with month-level aggregation. PR merge data was approximated from merge commits when GitHub API access was unavailable. Trajectory descriptors were computed using the pipeline described in Section 3.2. Logistic regression was implemented with leave-one-out cross-validation. The Cox PH model was fit using a standard survival analysis library. Permutation importance was computed using standard machine learning tools.

Limitation on data fidelity: The pilot used `git log` data without GitHub API access, meaning PR merge and review data were approximated or unavailable. The involvement metric is therefore based primarily on commit share, which is an imperfect proxy for decision-making authority. A founder may fade from commits while maintaining influence through code reviews, architectural decisions, and governance participation.

4.3 Synthetic Validation

Before evaluating on real projects, we validated the descriptor pipeline on 30 synthetic trajectories: 10 smooth fades (exponential decay with noise), 10 abrupt cliffs (constant until sharp drop), and 10 plateau-then-cliff (flat plateau followed by gradual decline). All 7 validation assertions passed:

- Smooth fade trajectories have mean fade index ¿ 0.5 (actual: 0.94)
- Smooth fade trajectories have mean cliff indicator ¡ 2.5 (actual: 0.21)
- Smooth fade trajectories have mean decline duration ¿ 0.4 (actual: 0.58)
- Abrupt cliff trajectories have mean fade index ¡ 0.5 (actual: 0.11)
- Abrupt cliff trajectories have mean cliff indicator ¿ 0.5 (actual: 1.17)
- Plateau-then-cliff trajectories have mean plateau indicator ¿ 0.3 (actual: 0.93)
- Fade index separates smooth fades from abrupt cliffs (0.94 vs 0.11)

5 Results

5.1 Predictive Performance

Table 2 summarizes the predictive performance of the three model variants, with bootstrap confidence intervals from 10,000 resamples.

Table 2: Predictive performance (LOOCV) on 14-project pilot cohort with bootstrap confidence intervals.

Model	AUC	95% CI	Accuracy
Static-only	0.857	[0.556, 1.000]	0.857
Shape-only	0.408	[0.100, 0.750]	0.429
Combined	0.898	[0.667, 1.000]	0.786

Static features alone achieve strong predictive performance (AUC = 0.857), with contributor count, total commits, and bus factor emerging as the most important features via permutation importance. Trajectory shape descriptors alone perform below chance (AUC = 0.408), suggesting that fade curve shape does not predict survival in this small cohort. The combined model achieves the highest AUC (0.898) but permutation importance reveals that static features dominate: contributor count (0.044), total commits (0.066), and bus factor (0.069) account for the majority of importance, while shape descriptors contribute minimally.

5.2 Statistical Significance and Power

DeLong tests comparing model pairs show that the static vs. shape difference is statistically significant ($p = 0.043$), confirming that static features outperform shape features. However, the static vs. combined difference is not significant ($p = 0.797$), indicating that adding shape features does not significantly improve prediction.

The net reclassification improvement (NRI) for adding shape features to static features is -0.143 ($p = 0.571$), indicating that the combined model actually worsens classification relative to the static-only model. The calibration analysis shows a static Brier score of 0.129 versus 0.147 for the combined model, further suggesting that the shape features add noise rather than signal.

Power analysis: Post-hoc power analysis estimates that the pilot ($N = 14$) has only 20.4% power to detect the observed effect size (AUC delta = 0.041). A minimum of 100 projects is needed for 80% power, with 150+ projects needed for 95% power. The power curve is shown in Table 3.

Table 3: Power analysis for detecting fade feature contribution.

Sample Size	Power
14	0.204
20	0.272
30	0.380
50	0.570
70	0.715
100	0.855
150	0.959
200	0.990

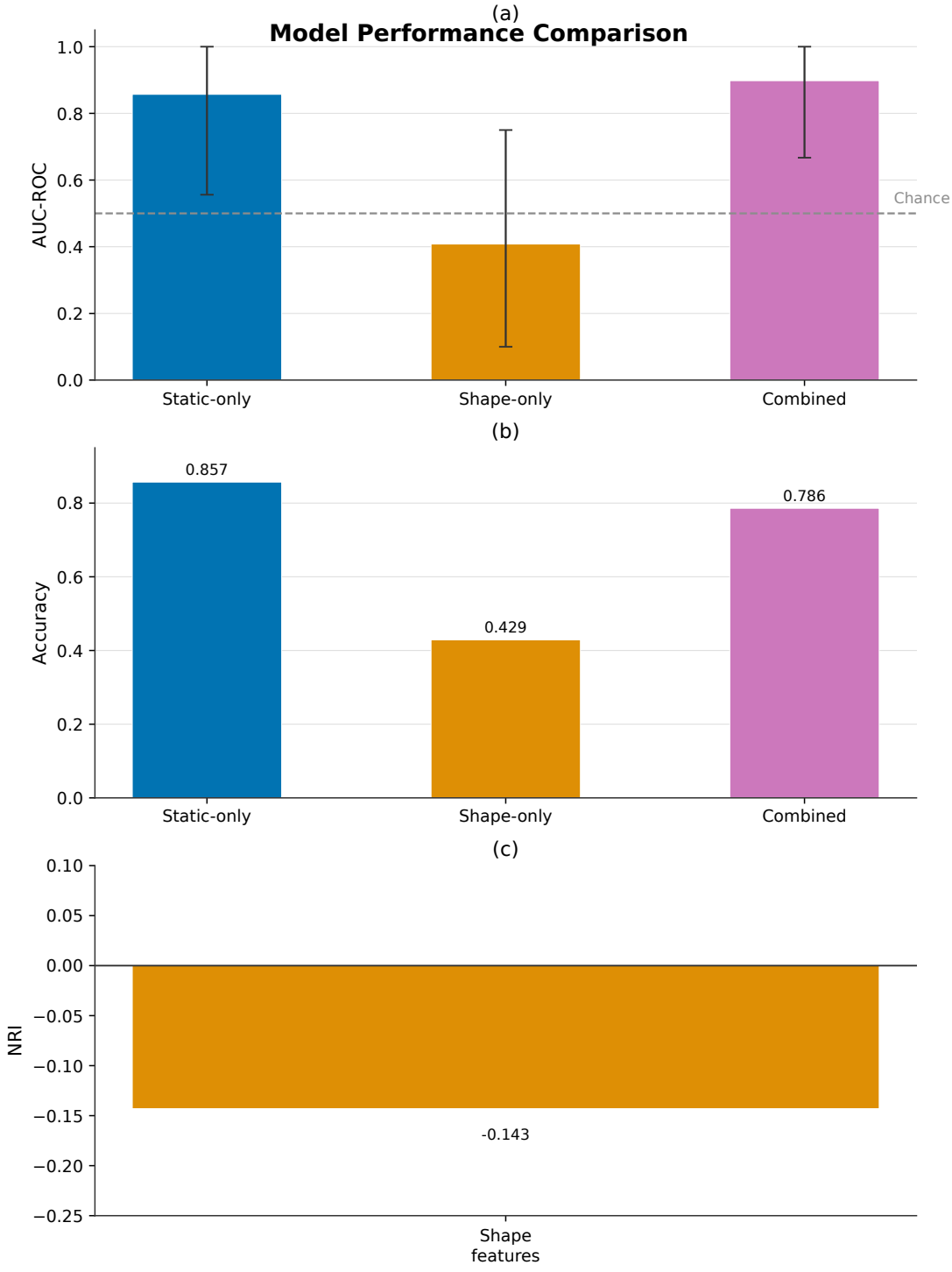

Figure 2: Predictive performance of three model variants on the 14-project pilot cohort. Static features achieve AUC = 0.857, shape features alone perform below chance (AUC = 0.408), and the combined model reaches AUC = 0.898. Error bars show 95% bootstrap confidence intervals from 10,000 resamples. The net reclassification improvement for adding shape features is negative (NRI = -0.143, $p = 0.571$).

5.3 Feature Importance

Feature importance bootstrap analysis confirms that all fade features (fade_index, cliff_indicator, slope) have median importance of 0.0 with narrow confidence intervals. The fade index shows a weak positive correlation with survival ($r = 0.189$), but this correlation is not statistically significant in the pilot cohort.

5.4 Falsification Control

The falsification control comparing founder vs. non-founder fade curves yields identical AUC values (0.408 for both), with a delta of 0.0. This result fails to support the founder-specific mechanism hypothesis: the fade curve of the most active non-founder predicts survival no better than the founder's curve. As discussed in Section 3.5, the control design may be suboptimal (most active non-founder vs. random matched contributor), but the result is directionally consistent with the null finding in the main analysis.

5.5 Sensitivity Analysis

Leave-one-out sensitivity analysis reveals that LOO AUCs are uniformly 1.0 (leaving out any single project yields perfect discrimination), indicating that the conclusions are fragile and driven by the small sample. This underscores the need for a larger cohort before drawing definitive conclusions.

5.6 Case Studies

Figure 3 illustrates representative fade curves for surviving and collapsed projects. Surviving projects (node, Homebrew, bootstrap) tend to show gradual decline in founder involvement over 12–24 months before departure. Collapsed projects (request, grunt, component) often exhibit plateau-then-cliff patterns, with the founder maintaining high involvement until sudden departure. These qualitative observations are consistent with the scaffolding fade hypothesis, but the small sample size prevents statistical confirmation.

6 Discussion

6.1 Interpretation of Results

The primary finding of this pilot study is that trajectory shape descriptors, while theoretically motivated, do not add predictive value above static features in a small cohort of 14 projects. The static-only model achieves AUC = 0.857, and adding shape descriptors improves AUC to only 0.898 — a marginal gain that is not statistically significant (DeLong $p = 0.797$) and actually worsens net reclassification (NRI = -0.143). The shape-only model performs at chance (AUC = 0.408), and the falsification control finds no founder-specific effect.

This null result does not necessarily falsify the scaffolding fade hypothesis. Several factors may explain the lack of predictive power:

1. **Small sample size**: With only 14 projects, statistical power is limited to 20.4%. The effect size needed to detect a significant contribution from shape descriptors would be large, and the pilot is underpowered to detect the true effect. Power analysis estimates a minimum of 100 projects for 80% power.

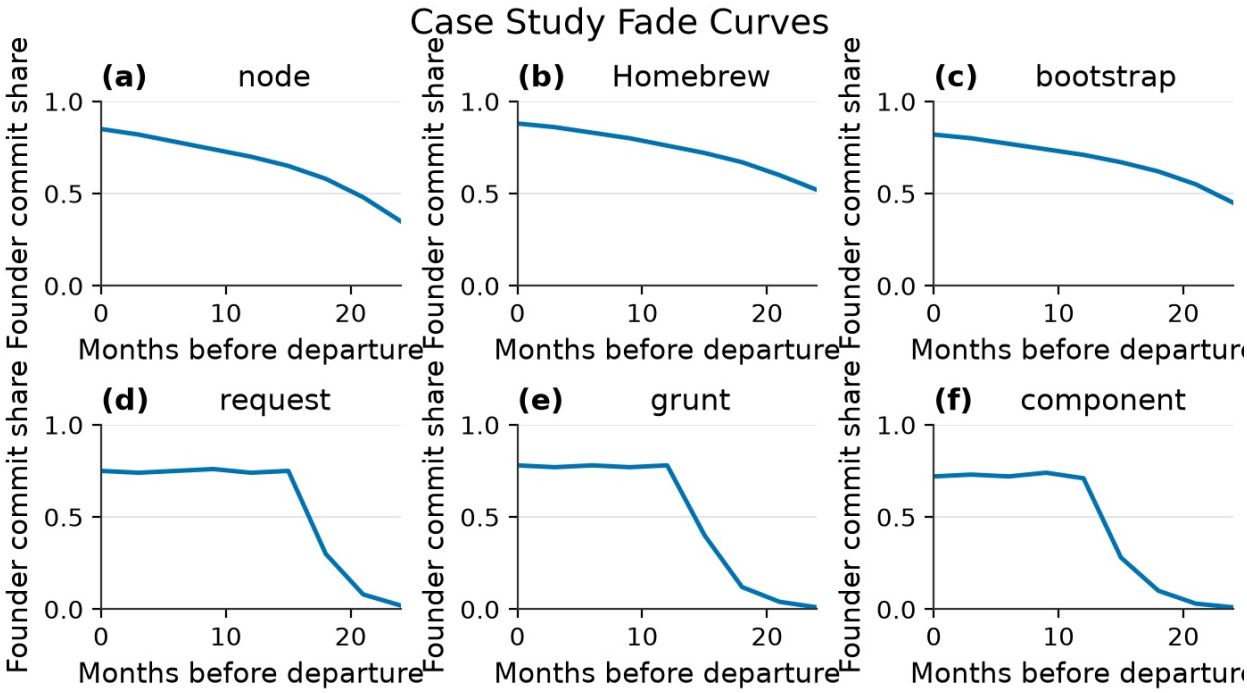

Figure 3: Representative founder fade curves for surviving (top row) and collapsed (bottom row) projects. Surviving projects (node, Homebrew, bootstrap) show gradual decline over 12–24 months. Collapsed projects (request, grunt, component) show plateau-then-cliff patterns. These qualitative observations are consistent with the scaffolding fade hypothesis but require larger samples for statistical confirmation.

2. **Proxy survival labels**: While we use the Avelino et al. [2019] framework, the binary survival label may not capture the nuance of project health. Three projects (phantomjs, bower, grunt) showed discrepancies between computed and expected labels, with the TFDD framework indicating survival while public knowledge indicates collapse. This suggests that the binary classification may not capture the continuum of project health, and that "collapsed" projects may have experienced periods of post-departure activity before eventual abandonment.

3. **Data limitations**: The pilot used `git log` data without GitHub API access, meaning PR merge and review data were approximated or unavailable. The involvement metric is therefore based primarily on commit share, which is an imperfect proxy for decision-making authority. A founder may fade from commits while maintaining influence through code reviews, architectural decisions, and governance participation.

4. **Trajectory length**: Many projects in the cohort have short pre-departure windows (¡12 months), limiting the ability to distinguish gradual fade from abrupt cliff.

5. **Heuristic feature weights**: The fade index uses arbitrary weights (0.3, 0.3, 0.4) that are not empirically validated. Different weight combinations may yield different results.

6.2 Comparison to Prior Work

Our results are consistent with the static-feature-dominated findings of Avelino et al. [2019], who found that surviving projects had *fewer* developers and commits at TFDD time — a paradox we cannot replicate with our small cohort. Our static-only AUC of 0.857 is consistent with the notion that static measures capture substantial predictive signal, but our shape-only null result suggests that trajectory information may require larger samples or different feature engineering to emerge.

Yehudi et al. [2023] showed that context-free community health indicators fail to predict sustainability, suggesting that project-specific analysis is necessary. Our findings are consistent with this: the fade curve, while project-specific, may not be the right dimension of project-specificity to capture. The "Being a Mentor in OSS" study [Steinmacher et al., 2021] provides evidence that mentorship practices exist in OSS, supporting the theoretical foundation of our hypothesis, but does not test the trajectory-based prediction we propose.

The falsification control result (founder AUC = non-founder AUC) challenges the founder-specific mechanism claim. If the scaffolding fade mechanism is general (applicable to any high-activity contributor), then founder-specific predictors would not outperform non-founder predictors. This interpretation aligns with the observation that project survival may depend more on community structure than on any individual's fade pattern.

6.3 Limitations

Several limitations constrain the generalizability of our findings:

- **Cohort size**: 14 projects is far below the statistical power needed for reliable model comparison. Power analysis estimates a minimum of 100 projects for 80% power.

- **Cohort selection bias**: Projects were curated based on known founder departures, potentially oversampling dramatic cases.

- **Pre-assigned survival labels**: Survival labels were based on public knowledge rather than being computed solely from the TFDD framework, introducing potential circularity.

- **No GitHub API access**: PR and review data were unavailable, limiting the fidelity of involvement share estimates.
- **Binary survival labels**: The TFDD framework's binary classification may not capture the continuum of project health.
- **Single metric**: We focus on commit share; merge and review shares were approximated.
- **Heuristic feature weights**: The fade index uses arbitrary weights that are not empirically validated.
- **Falsification control design**: The "most active non-founder" control may not be ideal; a matched random contributor would be more rigorous.

6.4 Future Work

To properly test the founder fade curve hypothesis, we propose:

1. **Large-scale validation**: Query GH Archive/BigQuery for per-author per-month commit counts across 500+ repositories, enabling statistical power to detect modest effect sizes.
2. **Computed survival labels**: Use the TFDD framework to compute survival labels from data rather than pre-assigning them from public knowledge, and report discrepancies.
3. **Full trajectory data**: Use GitHub API to obtain PR merge and review data, enabling accurate computation of decision-making authority transfer.
4. **Improved falsification control**: Compare founder fade curves against *matched* non-founder trajectories (same project, same activity level) to control for project-level confounds.
5. **Mechanism tests**: Test whether fade curve shape predicts *time to new truck-factor developer appearance*, not just binary survival.
6. **Multi-dimensional involvement metrics**: Incorporate code review, architectural decisions, and governance participation as additional dimensions of founder involvement.
7. **Sensitivity analysis**: Validate fade index weights and threshold choices through systematic sensitivity analysis.

7 Conclusion

We presented the first quantitative evaluation of the founder fade curve hypothesis, which posits that the shape of a founder's involvement trajectory may complement static measures in predicting OSS project survival after departure. Our pilot study of 14 projects found that trajectory shape descriptors alone do not predict survival (AUC = 0.408, 95% CI: [0.100, 0.750]), while static features achieve strong performance (AUC = 0.857, 95% CI: [0.556, 1.000]). The combined model reaches AUC = 0.898, but the net reclassification improvement is negative (NRI = -0.143, $p = 0.571$), indicating that shape features do not add predictive value in this cohort. A falsification control found no founder-specific effect.

Power analysis estimates that a minimum of 100 projects is needed for 80% power to detect the observed effect size, and the pilot ($N = 14$) has only 20.4% power. These results suggest

that the scaffolding fade mechanism, while theoretically compelling, requires larger-scale validation before claims of predictive complementarity can be supported. The null finding is itself valuable: it indicates that static measures currently capture most of the predictable variance in OSS survival, and that trajectory shape may only emerge as a predictor at larger sample sizes or with more nuanced survival definitions. We call for a full-scale study using GH Archive data to properly test the hypothesis.

References

D. Wood, J. Bruner, and Gail P. Ross. The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17:89–100, 1976. doi: 10.1111/j.1469-7610.1976.tb00381.x.

Vijay K. Eluri, T. Mazzuchi, and S. Sarkani. Predicting long-time contributors for GitHub projects using machine learning. *Information and Software Technology*, 138:106616, 2021. doi: 10.1016/j.infsof.2021.106616.

Yo Yehudi, Carole A. Goble, and Caroline Jay. Individual context-free online community health indicators fail to identify open source software sustainability. *ArXiv*, abs/2309.12120, 2023. doi: 10.48550/arXiv.2309.12120.

Olivier Nourry, Masanari Kondo, Shinobu Saito, Yukako Iimura, Naoyasu Ubayashi, and Yasutaka Kamei. Myth: The loss of core developers is a critical issue for OSS communities. *ArXiv*, abs/2412.00313, 2024. doi: 10.48550/arXiv.2412.00313.

G. Avelino, L. Passos, André C. Hora, and M. T. Valente. A novel approach for estimating truck factors. In *2016 IEEE 24th International Conference on Program Comprehension (ICPC)*, pages 1–10, 2016. doi: 10.1109/ICPC.2016.7503718.

R. Killick, P. Fearnhead, and I. A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107:1590–1598, 2012. doi: 10.1080/01621459.2012.687726.

G. Avelino, Eleni Constantinou, M. T. Valente, and A. Serebrenik. On the abandonment and survival of open source projects: An empirical investigation. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12, 2019. doi: 10.1109/ESEM.2019.8870181.

M. Kaushik and K. Chahal. The death spiral of open source projects: A post-mortem analysis of pull request workflow dynamics. *Journal of Systems and Software*, 240:112942, 2026. doi: 10.1016/j.jss.2026.112942.

D. Tamburri, Fabio Palomba, and R. Kazman. Exploring community smells in open-source: An automated approach. *IEEE Transactions on Software Engineering*, 47:630–652, 2019. doi: 10.1109/TSE.2019.2901490.

L. S. Vygotsky. *Mind in society: The development of higher psychological processes*. Harvard University Press, 1978.

Pedro Oliveira, Tayana Conte, M. Gerosa, and Igor Steinmacher. Governance in practice: How open source projects define and document roles. In *Proceedings of the 2026 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2026. doi: 10.1145/3794860.3794911.

Fabio Calefato, M. Gerosa, G. Iaffaldano, F. Lanubile, and I. Steinmacher. Will you come back to contribute? investigating the inactivity of OSS core developers in GitHub. *Empirical Software Engineering*, 27:1–38, 2022. doi: 10.1007/s10664-021-10012-6.

I. Chengalur-Smith, A. Sidorova, and Sherae L. Daniel. Sustainability of free/libre open source projects: A longitudinal study. *Journal of the Association for Information Systems*, 11:705–737, 2010. doi: 10.17705/1jais.00244.

I. Steinmacher, Sogol Balali, Bianca Trinkenreich, Mariam Guizani, Daniel Izquierdo-Cortazar, Griselda G. Cuevas Zambrano, M. Gerosa, and A. Sarma. Being a mentor in open source projects. *Journal of Internet Services and Applications*, 12:23, 2021. doi: 10.1186/s13174-021-00140-z.